

Project Documentation: Student Performance Analytics System

1. Project Title

- **Project Name:** Student Performance Analytics System
- **Student Name:** Adib Muzammil Ali
- **College Name:** G H Rasoni University Amravati, Maharashtra
- **Course / Batch:** Python with AI

2. Objective

The primary objective of this project is to build an automated data analysis tool that evaluates student academic performance. It solves the problem of manually calculating grades, tracking pass/fail statuses, and identifying class trends by automating the mathematical computations and logic using Python.

3. Technologies Used

- **Programming Language:** Python
- **Libraries:** Pandas (for data manipulation and structured data analysis) and NumPy (for fast numerical computations across arrays).
- **Tools:** Visual Studio Code, Git, and GitHub.

4. Dataset Description

A custom CSV dataset named `student_data.csv` was generated for this project.

- **Number of Records:** 20 student rows.
- **Important Columns:** `Student_ID`, `Name`, `Department`, `Math (Marks)`, `Physics (Marks)`, `Chemistry (Marks)`, and `Attendance (Percentage)`.

5. Implementation

The project was implemented in the following step-by-step manner:

- **Step 1 (Data Loading):** Used `pandas.read_csv()` to load the dataset into a `DataFrame` and check the structure using `.head()`.
- **Step 2 (Data Processing with NumPy):** Extracted the subject columns into a NumPy array to calculate the `Total_Marks` using `np.sum()` and Average using `np.mean()`.
- **Step 3 (Custom Functions & Iteration):** Created functions like `calculate_grade()` and

check_pass_fail(). Used df.iterrows() to loop through the DataFrame and apply these functions to determine each student's final grade and status based on strict rules (minimum 40 marks per subject and 75% attendance).

- **Step 4 (Data Analysis):** Utilized Pandas and NumPy aggregate functions to find the highest, lowest, and overall class averages, along with the best-performing subject.

6. Key Features

- **Automated Data Generation:** A helper function automatically creates a realistic 20-record dataset.
- **Dynamic Grading:** Assigns A, B, C, D, or F grades based on calculated averages.
- **Strict Pass/Fail Logic:** Evaluates both subject scores and attendance thresholds simultaneously.
- **Top Performer Identification:** Sorts the DataFrame to display the top 3 highest-scoring students.
- **Subject-wise Analytics:** Compares averages, highest, and lowest scores across individual subjects.

7. Output

Total records loaded: 20

Columns: Student_ID, Name, Department, Math, Physics, Chemistry, Attendance

Preview of first 3 rows:

	Student_ID	Name	Department	Math	Physics	Chemistry	Attendance
0	S101	Alice	Computer Science	85	80	78	85
1	S102	Bob	Computer Science	45	50	55	70
2	S103	Charlie	Computer Science	92	89	94	95

--- STEP 3: Processing Data ---

Calculations complete (Totals, Averages, Grades, Pass/Fail).

Processed Dataset Preview:

	Name	Total_Marks	Average	Grade	Status
0	Alice	243	81.000000	B	Pass
1	Bob	150	50.000000	F	Fail
2	Charlie	275	91.666667	A	Pass
3	David	235	78.333333	C	Pass
4	Eva	263	87.666667	B	Pass

--- STEP 4 & 5: Performance Analysis Results ---

1. OVERALL CLASS METRICS:

- Class Average: 75.55%
- Highest Average: 96.67%
- Lowest Average: 39.00%

2. PASS/FAIL STATISTICS:

- Passed: 16 students
- Failed: 4 students

3. TOP 3 PERFORMING STUDENTS:

- Paul (Information Tech) | Score: 96.67 | Grade: A
- Grace (Computer Science) | Score: 94.00 | Grade: A
- Charlie (Computer Science) | Score: 91.67 | Grade: A

4. SUBJECT-WISE PERFORMANCE:

- Math : Avg = 75.20, High = 98, Low = 32
- Physics : Avg = 75.50, High = 96, Low = 38
- Chemistry : Avg = 75.95, High = 97, Low = 42

5. HIGHEST SCORING SUBJECT:

- Chemistry with an average of 75.95

8. Final Outcome

The project successfully processed the raw student data and generated a clean, readable analytics report. It correctly identified the class average (75.55%), tracked that 16 students passed while 4 failed, and successfully highlighted the top 3 performing students. The data was structured cleanly back into the DataFrame.

9. Challenges & Learning

- **Challenges Faced:** Initially, connecting the local project folder to the remote GitHub repository using Git commands in the terminal was challenging due to unconfigured remote links.
- **How it was solved:** Solved this by properly initializing Git, adding a .gitignore file to avoid uploading unnecessary virtual environment files, and linking the remote origin URL using the git remote add command.
- **What was learned:** Gained hands-on experience in using Pandas DataFrames for

structured data, performing vector computations with NumPy, and managing code version control with Git and GitHub.

10. GitHub Repository

- **Repository Link:** <https://github.com/adibali05/Student-Performance-Analytics-System->